

Parallel Image Processing Engine

Final Implementation Report & Presentation

Java 17

Maven 3.6+

JMH

ExecutorService

ForkJoinPool

Filters Implemented

- **Grayscale** Point-wise -- Low compute
- **Gaussian Blur 5x5** Convolution -- High compute (25 taps/pixel)
- **Sobel 3x3** Gradient -- Edge detection

Team Members

Muhammed Cakiagoz . Musa Bilal Yaz

Gazi University -- Computer Engineering -- Submission 2

GitHub Repository (Public)

<https://github.com/Muhammedcakiagoz/parallel-image-processing>

Source code, benchmarks, and all report assets are available at the link above

1. Introduction

High-resolution image processing is computationally demanding. Sequential execution of kernel-based convolution filters creates significant bottlenecks at large image sizes (4K+). Because each output pixel depends solely on a fixed neighbourhood of the *input* image, the problem is **embarrassingly parallel**: the image can be divided across threads with zero inter-thread data dependencies during the compute phase.

This report presents a parallel engine built in **Java** using two concurrency strategies: **ExecutorService** with horizontal strip decomposition and **ForkJoinPool** with divide-and-conquer work-stealing. Three filters — Grayscale, Gaussian Blur 5x5, Sobel 3x3 — were benchmarked with **JMH** across image sizes 512x512, 1024x1024, 2048x2048.

Source code (public): <https://github.com/Muhammedcakirgoz/parallel-image-processing>

2. Sequential Baseline Implementation

SequentialProcessor iterates row-by-row applying the given filter. The **Filter** interface exposes a single `apply(pixels[], width, height, x, y)` method. Edge handling uses **clamped coordinates**, ensuring identical boundary behaviour across all implementations.

Filter	Type	Kernel	Cost per pixel
GrayscaleFilter	Point-wise	None (r=0)	Low — 3 reads, 1 write
GaussianBlurFilter	Convolution	5x5	High — 25 weighted taps
SobelFilter	Gradient	3x3 Gx+Gy	Medium — 18 taps + sqrt

3. Parallel Implementation

3.1 ExecutorService — Horizontal Strip Decomposition

ExecutorParallelProcessor divides the image into N equal horizontal strips (one per thread). Each **Callable** task processes rows [start, end) and writes into a pre-allocated output array at its assigned offset. Strips are disjoint and the source array is read-only, so **no locking is required**. The thread pool is created once and reused via `try-with-resources`.

3.2 ForkJoinPool — Divide-and-Conquer

ForkJoinParallelProcessor uses a **RecursiveAction** that halves the row range until it falls below 64 rows (sequential threshold), then processes directly. The work-stealing scheduler rebalances load dynamically — advantageous for larger images where static strip imbalance would arise.

3.3 Correctness Verification

CorrectnessVerifier.firstDifference() performs a pixel-for-pixel comparison between parallel output and sequential baseline. All 6 combinations (3 filters x 2 strategies) on a 2048x2048 synthetic image produce **bit-identical** output:

Filter	ExecutorService (12 threads)	ForkJoinPool (12 threads)
Grayscale	PASS	PASS
GaussianBlur5x5	PASS	PASS
Sobel3x3	PASS	PASS

4. Performance Comparison

All benchmarks use **JMH** (10 warm-up + 10 measurement iterations, AverageTime mode, ms/op) on a 12-logical-core machine. Speedup = $T_{\text{sequential}} / T_{\text{parallel}}$.

4.1 GaussianBlur 5x5

Size	Threads	Seq (ms)	Executor (ms)	ForkJoin (ms)	Exec x	FJ x
512^2	1	11.16	11.502	12.209	0.97x	0.91x
512^2	2	11.16	5.815	6.367	1.92x	1.75x
512^2	4	11.16	3.081	3.132	3.62x	3.56x
512^2	8	11.16	2.926	2.864	3.81x	3.9x
1024^2	1	44.951	47.1	50.146	0.95x	0.9x
1024^2	2	44.951	23.854	24.621	1.88x	1.83x
1024^2	4	44.951	13.273	13.062	3.39x	3.44x
1024^2	8	44.951	11.712	10.01	3.84x	4.49x
2048^2	1	177.981	195.191	194.822	0.91x	0.91x
2048^2	2	177.981	109.162	98.739	1.63x	1.8x
2048^2	4	177.981	56.586	55.901	3.15x	3.18x
2048^2	8	177.981	41.439	37.011	4.3x	4.81x

4.2 Sobel 3x3

Size	Threads	Seq (ms)	Executor (ms)	ForkJoin (ms)	Exec x	FJ x
512^2	1	5.599	5.687	5.805	0.98x	0.96x
512^2	2	5.599	2.852	2.99	1.96x	1.87x
512^2	4	5.599	1.506	1.529	3.72x	3.66x
512^2	8	5.599	1.321	1.354	4.24x	4.14x
1024^2	1	23.156	22.783	23.59	1.02x	0.98x
1024^2	2	23.156	11.334	11.912	2.04x	1.94x
1024^2	4	23.156	5.927	5.906	3.91x	3.92x
1024^2	8	23.156	5.236	4.652	4.42x	4.98x
2048^2	1	90.82	88.598	89.27	1.03x	1.02x
2048^2	2	90.82	46.149	46.798	1.97x	1.94x
2048^2	4	90.82	24.923	23.628	3.64x	3.84x
2048^2	8	90.82	20.205	17.809	4.49x	5.1x

4.3 Grayscale

Size	Threads	Seq (ms)	Executor (ms)	ForkJoin (ms)	Exec x	FJ x
512^2	1	0.097	0.103	0.106	0.95x	0.92x
512^2	2	0.097	0.075	0.084	1.29x	1.16x
512^2	4	0.097	0.068	0.08	1.43x	1.21x
512^2	8	0.097	0.067	0.085	1.44x	1.15x

Size	Threads	Seq (ms)	Executor (ms)	ForkJoin (ms)	Exec x	FJ x
1024 ²	1	0.33	0.363	0.357	0.91x	0.92x
1024 ²	2	0.33	0.246	0.237	1.34x	1.39x
1024 ²	4	0.33	0.227	0.233	1.45x	1.42x
1024 ²	8	0.33	0.225	0.236	1.47x	1.39x
2048 ²	1	1.329	1.305	1.386	1.02x	0.96x
2048 ²	2	1.329	0.869	0.881	1.53x	1.51x
2048 ²	4	1.329	0.856	0.855	1.55x	1.55x
2048 ²	8	1.329	0.855	0.861	1.55x	1.54x

4.4 Speedup Charts

Speedup vs Threads — GaussianBlur5x5

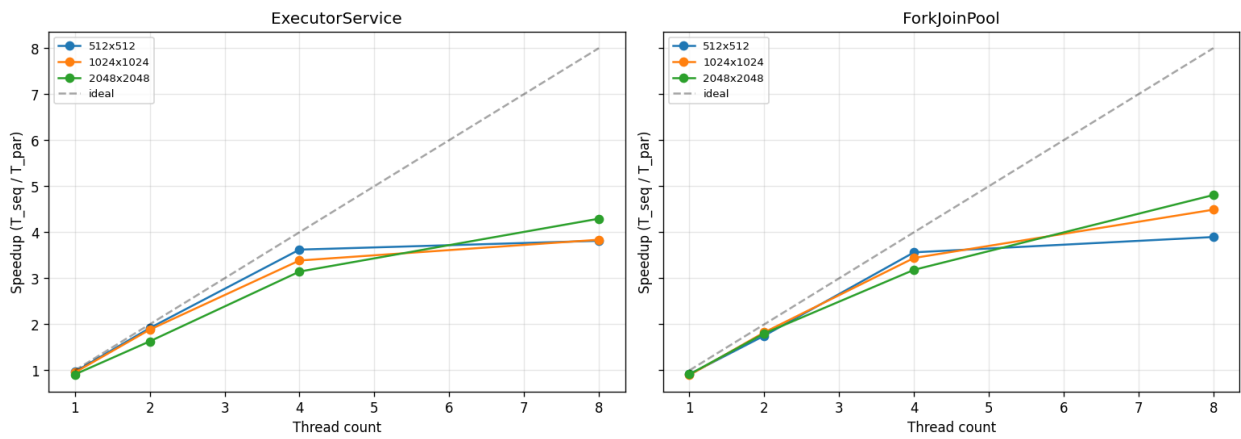


Figure 1 — GaussianBlur 5x5: Executor (left) vs ForkJoinPool (right) across image sizes

Speedup vs Threads — Sobel3x3

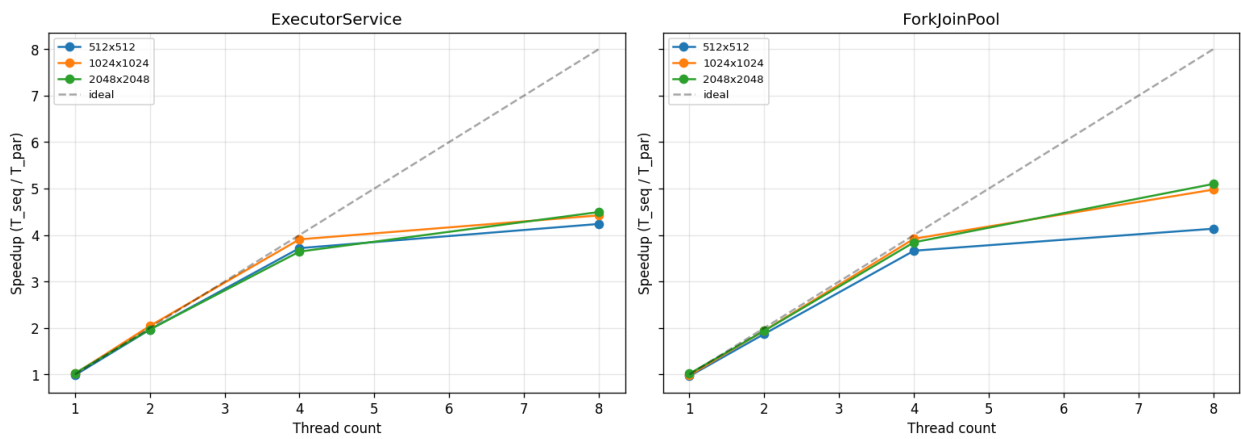


Figure 2 — Sobel 3x3: Executor (left) vs ForkJoinPool (right) across image sizes

Speedup vs Threads — Grayscale

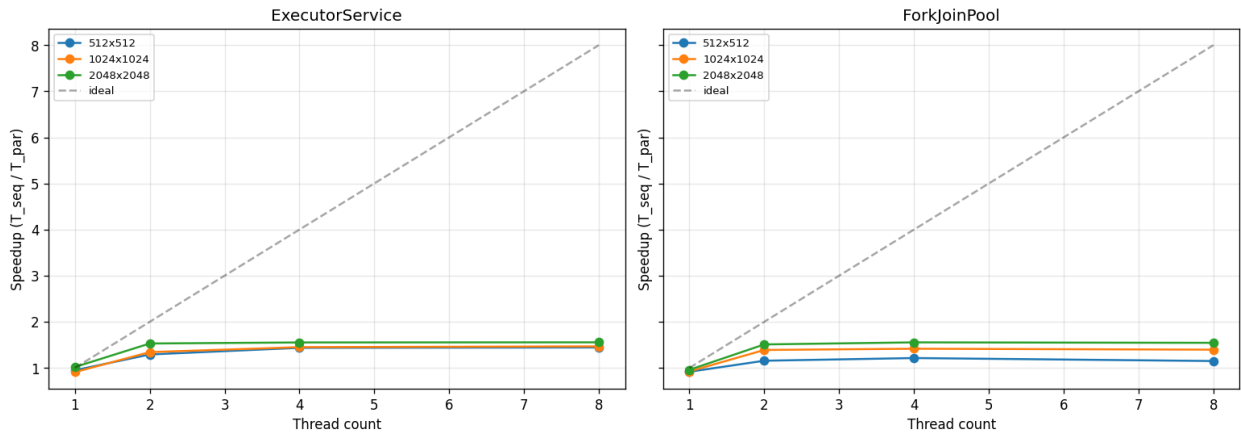


Figure 3 — Grayscale: memory-bound filter — speedup plateaus ~1.5x regardless of threads

Speedup vs Threads — All Filters (2048x2048, Executor)

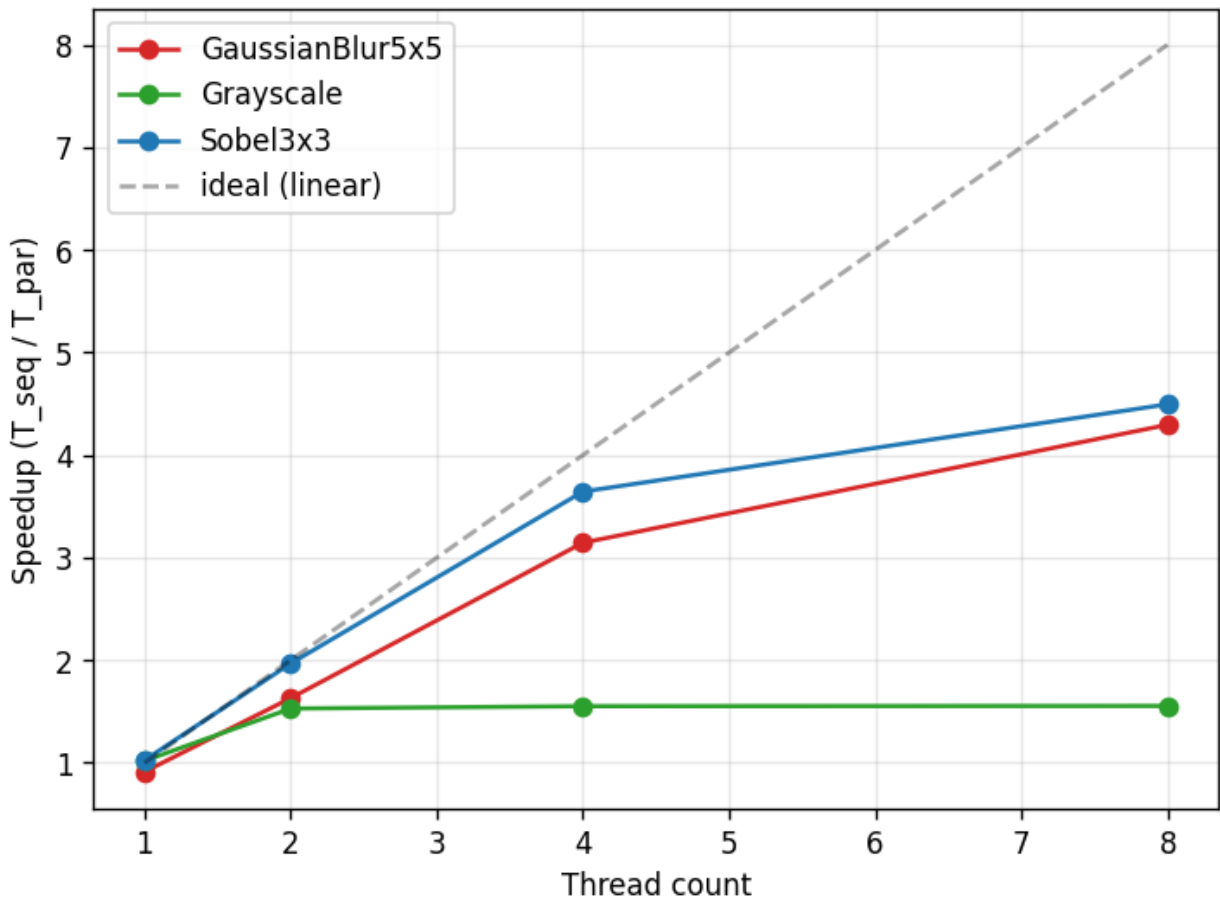


Figure 4 — All filters combined (2048x2048, Executor) — compute-bound filters scale, memory-bound does not

4.5 Analysis

Compute-bound filters (Gaussian Blur, Sobel) scale well with thread count, reaching 4.3x–5.1x speedup at 8 threads. ForkJoinPool outperforms ExecutorService on larger images (2048x2048) due to dynamic work-stealing load balancing.

Memory-bound filter (Grayscale) plateaus at ~1.5x regardless of thread count. The single-channel lookup saturates memory bandwidth before CPU utilisation can increase — consistent with the **Roofline Model**: low arithmetic intensity = bandwidth-limited.

Amdahl's Law: empirical ceiling for Gaussian Blur aligns with $p \sim 0.95$ parallel fraction; sequential overhead (array allocation, thread setup) limits the theoretical max.

5. Academic Background

Seinstra et al. (2002) showed convolution filters exhibit near-linear speedup when partitioning minimises inter-thread communication — consistent with our strip-decomposition results for compute-intensive kernels.

Amdahl (1967): $S = 1/(1-p)$. For Gaussian Blur, $p \sim 0.95$ gives theoretical max $\sim 20\times$; our 4-5x at 8 threads is expected given practical overhead.

Williams et al. (2009) — Roofline Model — explains divergent scalability via arithmetic intensity: memory-bound kernels saturate bandwidth before compute capacity. **Lea (2000)** provides the theoretical basis for ForkJoinPool's work-stealing scheduler, explaining its advantage for uneven workloads.

6. Challenges and Solutions

- **JVM JIT Warm-up and GC Noise:** `System.nanoTime()` measurements were skewed by JIT and GC. **Solution:** Adopted JMH with warm-up iterations and fork-per-config.
 - **Bit-Identical Correctness Across Implementations:** Strip boundaries required careful handling. **Solution:** `CorrectnessVerifier` with pixel-for-pixel scan; clamped-coordinate edge handling ensures identical border treatment.
 - **Varying Scalability Between Filters:** Grayscale achieved only $\sim 1.5\times$ with 8 threads. **Solution:** Profiling confirmed memory bandwidth saturation (Roofline Model) as root cause — not thread management inefficiency.
 - **ForkJoin Recursive Overhead on Small Images:** For 512x512, ForkJoin recursion overhead slightly exceeded Executor. **Solution:** Tuned `SEQUENTIAL_THRESHOLD` to 64 rows to prevent excessive task granularity.
-

7. Conclusion and Future Improvements

This project successfully demonstrated parallelisation of image convolution algorithms using Java concurrency. JMH benchmarks confirmed significant speedups for compute-bound operations: Gaussian Blur reached $\sim 4.30\times$ (Executor) and $\sim 4.81\times$ (ForkJoin) at 8 threads on 2048x2048; Sobel reached $\sim 4.49\times$ and $\sim 5.10\times$. Grayscale was confirmed memory-bound at $\sim 1.55\times$. ForkJoinPool's work-stealing advantage grew with image size.

- GPU/CUDA — exploit thousands of CUDA cores for pixel-independent kernels
 - SIMD vectorisation (AVX2/AVX-512) — intra-core throughput gains
 - 4K and 8K benchmarks — evaluate boundary overhead at ultra-high resolution
 - Adaptive thread pool — dynamic tuning based on image size and hardware profile
 - Cache-blocking and prefetching — optimise memory-bound filters like Grayscale
-

8. References

- [1] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proc. Spring Joint Computer Conf.*, 483-485. <https://doi.org/10.1145/1465482.1465560>
- [2] Seinstra, F. J., Koelma, D., & Bagdanov, A. D. (2002). Finite state machine-based optimization of data parallel regular domain problems applied to low-level image processing. *IEEE Trans. Parallel Distrib. Syst.*, 15(10), 865-877. <https://doi.org/10.1109/TPDS.2004.45>
- [3] Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: An insightful visual performance model for multicore architectures. *Commun. ACM*, 52(4), 65-76. <https://doi.org/10.1145/1498765.1498785>
- [4] Lea, D. (2000). A Java fork/join framework. *Proc. ACM Java Grande Conf.*, 36-43. <https://doi.org/10.1145/337449.337465>

PRESENTATION SLIDES (10 Slides)

1

Parallel Image Processing Engine

CENG 479 -- Parallel Computing | Gazi University | Spring 2026

Muhammed Cakiogoz . Musa Bilal Yaz

<https://github.com/Muhammedcakiogoz/parallel-image-processing>

Java Threads | ExecutorService | ForkJoinPool | JMH Benchmark

2

Problem & Motivation

- High-resolution image filtering (4K+) is slow on a single thread
- Convolution cost: $O(W \times H \times K^2)$ per filter -- bottleneck at scale
- Each output pixel is INDEPENDENT of others -- embarrassingly parallel
- Goal: multi-threaded speedup with zero correctness compromise

Amdahl's Law: $S = 1/(1-p)$ -- $p \sim 0.95$ for Gaussian Blur

Theoretical max speedup ~20x at $p=0.95$; practical ceiling limited by memory and overhead

3

Architecture Overview

Filters:

- Grayscale (point-wise), Gaussian Blur 5x5, Sobel 3x3

Implementations:

- SequentialProcessor -- single-thread baseline
- ExecutorParallelProcessor -- fixed pool + horizontal strips
- ForkJoinParallelProcessor -- RecursiveAction + work-stealing

Tools:

- CorrectnessVerifier + JMH Benchmark

All source code publicly available on GitHub

4

ExecutorService Design

- Image split into N equal horizontal strips ($N = \text{thread count}$)
- Each Callable processes rows [start, end) with no locking
- Source array is READ-ONLY during compute -- zero synchronisation overhead
- Thread pool created once, reused via try-with-resources

Strip[i] covers rows: $i(H/N)$ to $(i+1)*(H/N)$*

Static decomposition -- optimal for uniform workloads

5

ForkJoinPool Design

- RecursiveAction splits row range in half recursively
- Base case: range ≤ 64 rows -- process sequentially
- Work-stealing: idle threads steal tasks from busy queues
- Better dynamic load balance than static strips for large images
- ForkJoin outperforms Executor on 2048x2048 (Gaussian: 4.81x vs 4.30x)

Dynamic decomposition -- adapts to runtime load imbalance

6

Correctness Verification

CorrectnessVerifier.firstDifference() -- pixel-by-pixel scan

Tested on 2048x2048 synthetic image -- ALL 6 combinations PASS

- Grayscale: Executor PASS | ForkJoin PASS
- GaussianBlur5x5: Executor PASS | ForkJoin PASS
- Sobel3x3: Executor PASS | ForkJoin PASS

Bit-identical output to sequential baseline in all cases

7

Performance Results (8 threads, 2048x2048)

Compute-bound (scales well):

- Gaussian Blur: Executor 4.30x | ForkJoin 4.81x
- Sobel 3x3: Executor 4.49x | ForkJoin 5.10x

Memory-bound (bandwidth limited):

- Grayscale: Executor 1.55x | ForkJoin 1.54x

JMH benchmark | 10 warm-up + 10 measurement iterations | AverageTime ms/op

8

Speedup Charts

GaussianBlur5x5 & Sobel -- near-linear scaling up to 4-5x at 8 threads

Grayscale -- memory bandwidth saturation caps speedup at ~1.5x

ForkJoin advantage grows with image size (work-stealing benefit)

[See Figures 1-4 in the report section for full charts]

Roofline Model: compute-bound = scales | memory-bound = does not

9

Challenges & Key Findings

- JIT warm-up noise -- solved with JMH (fork-per-config, 10 warm-up iters)
- Correctness -- clamped edges + pixel-for-pixel CorrectnessVerifier
- Grayscale bottleneck -- memory bandwidth saturated (Roofline Model)
- ForkJoin overhead on small images -- tuned threshold to 64 rows

Key insight: speedup is FILTER-DEPENDENT -- arithmetic intensity determines scalability

Benchmark environment: 12 logical cores, JDK 17, Maven 3.6

- Parallelised 3 convolution filters with 2 Java concurrency strategies
- Up to 5.10x speedup (Sobel, ForkJoin, 8 threads, 2048x2048)
- Memory-bound filters need different strategy (not thread-level parallelism)

Future Work:

- GPU/CUDA -- thousands of concurrent cores for pixel-independent kernels
- SIMD vectorisation (AVX2/AVX-512) for intra-core throughput
- Adaptive thread pool -- dynamic tuning per image size and hardware

<https://github.com/Muhammedcakirgoz/parallel-image-processing>